

AI 协作反思日志 — Phase 3 详细设计

团队：第67组 日期：2026/5/14

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量 (1-5)	人工修改幅度 (%)
核心类图设计	Codex	基于 sec-ii-2026 远程仓库、P0/P1/P2 阶段文档和当前代码结构，为 CampusHub 设计核心类图，要求覆盖用户、需求、订单、评价、通知、论坛、管理等模块，标注关键属性、方法、类之间关系，并体现状态模式和策略模式；同时在文档不一致的情况下收敛到 Vue + Spring Boot + 单体分层的最终主线	4	35%
API补全并整理为文档	ChartGPT	基于我给出的类图和API接口，补全接口使用逻辑并为每一个接口按照要求添加请求体以及失败、成功样例，保持简洁格式避免冗余内容	5	10%
SOLID检查清单	Deepseek	以“资深软件架构师与领域建模专家”为角色定位，向 AI 提供 CampusHub 项目的完整上下文（7个功能模块、分层单体架构约束、4人团队10周交付、非功能需求），要求输出标准 UML 类图，涵盖实体层、服务层、数据访问层的属性与方法、枚举定义、类间关系及模块依赖简图，并强调服务层应依赖接口而非具体实现、实体类职责分离、枚举完整枚举等设计要点。	4	20%
建表SQL生成	ChartGPT	根据E-R图例与API接口中展示出的的不同实体之间的关系以及所拥有的属性，生成建表SQL语句，确保遵守不同对象之间的数量关系并且不要自己建立其他的无关表	5	15%

2. AI 助力分析（Q1）

AI 最大的助力体现在：它充当了“设计草稿的高效产出者”与“设计缺陷的精准暴露者”这一双重角色，将团队的精力从繁琐的文档撰写转移到了更具价值的批判性审查与设计决策上。具体而言，AI 能依据需求文档和架构约束，快速生成包含类图、API 定义、数据库 SQL 的结构化初稿，极大压缩了从零到一的时间。然而其更大的价值在于——它产出的初稿往往存在典型的“教科书式”缺陷，例如违反 SOLID 原则（如类职责过重、高层依赖具体实现）、API 设计缺乏错误码处理或参数校验不完整等。这让团队不再是被动背诵原则，而是通过主动“捉虫”AI 生成的具体设计，在反复的审查和修正中，将抽象的设计原则内化为实在的工程判断力，完成了从“形式上的实践”到“认知上的掌握”的转化。

3. AI 误导分析（Q2）

- **SOLID 检查中发现的 AI 设计问题数量：** 4

- **最严重的设计问题是：** 依赖倒转原则违反，高层模块直接依赖底层具体实现，严重破坏架构松耦合与可替换性

4. Prompt 改进计划（Q3）

- **上阶段改进措施的验证结果：** 在给出输出重点并提醒AI使用简洁格式的情况下，AI输出质量有明显进步，生成物距离我们想要的输出更加接近，尤其是在输出格式明确给出的情况下，AI帮助我们生成的文档几乎可以直接用来使用
- **本阶段新的改进措施：** 一些非文字性的输出需求，AI有时候无法精准地意识到生成物图例生成的规则，可以通过给出一些简单的例子确保AI能够按照你的想法来生成需要的图例

5. 本阶段的核心工程判断

- **决策内容：** P3 类图不直接按当前代码骨架逆向生成，而是按远程仓库 README、P1/P2 文档中最终收敛的业务主线来构建详细设计类图，并补入论坛模块、匿名发布/匿名评价等关键设计点。
- **为什么 AI 无法做这个决策：** 因为远程仓库中的 README、阶段文档和代码骨架之间存在一定不一致，个别文档里还保留了历史候选方案痕迹。AI 可以给出候选解释，但无法代替团队判断哪些内容属于当前有效设计、哪些只是历史残留。这个判断必须由人工结合仓库现状、课程交付目标和阶段任务要求来完成。